PropManage — Dissertation Review III

Viva Preparation Guide: Speaker Notes, Timing & Anticipated Questions

Companion document to PropManage_Dissertation_Review_III.pptx (23 slides). Target duration: 15-18 minutes presentation + 5-10 minutes Q&A.

How to Use This Guide

- Each slide entry gives you a natural sentence or two to open with ("Say:"), 2-4 bullet points you must land, and the questions a supervisor/panel is most likely to ask right after that slide, with a suggested answer grounded in what is actually implemented.
- The timings add up to roughly 15-16 minutes — this leaves buffer for panel interruptions and a comfortable pace. Do not rush; a calm 12-15 second pause per slide while the panel reads a chart or diagram is normal and expected.
- Practice the System Architecture and Algorithms slides most — they carry the highest technical weight and draw the most follow-up questions.
- If asked something outside this guide, it is always safe to say what is real: this is a working, evaluated MERN application with real security hardening and a genuine multilingual voice assistant — do not guess or overstate; say "that's a good direction for future work" where honest.

General Presentation Tips

- Open with energy on the title slide — state the project name and one-sentence hook: "a property management platform with verified payments, real-time alerts, and a multilingual voice assistant."
- Use the System Architecture diagram (Slide 12) as your anchor — if you get flustered anywhere, you can always return to it and re-orient the panel.
- When presenting numbers (38/38, 14/14, 100 users), say them slowly and clearly — panels remember concrete numbers more than adjectives.
- For the Execution slide, if screenshots are pasted in by viva day, briefly narrate the user journey (login -> search -> book -> pay -> voice assistant -> admin approval) rather than describing each screenshot in isolation.
- Close strong: the Conclusion slide's last line — a production-quality system built entirely on free-tier tooling — is a good note to land on before Future Scope.

Slide-by-Slide Notes

Slide 1 — Title *(0:20)*

Say: Good morning/afternoon. I'm presenting PropManage, a property management application built on the MERN stack with Power BI business intelligence, under the guidance of Dr. P. Kayal, Department of Information Technology.

Key points to emphasize:

- State your name and the project title clearly, once, and move on.
- Do not read the guide's designation/department aloud in detail — the panel can read it.

Slide 2 — Table of Contents *(0:20)*

Say: Here's the structure of today's presentation — I'll move through the problem, related work, the proposed architecture, implementation, and results, ending with a live look at the system and future scope.

Key points to emphasize:

- Don't read all 18 items aloud — just describe the 4-5 major movements (problem -> literature -> design -> results -> conclusion).

Slide 3 — Abstract *(0:45)*

Say: PropManage unifies the full property lifecycle — listing, booking, payments, contracts, and maintenance — for small and mid-size operators who today rely on spreadsheets and phone calls. It adds three things existing tools don't combine: cryptographically verified payments, real-time staff alerts, and a multilingual voice assistant.

Key points to emphasize:

- Emphasize "unifies" — the novelty is integration, not any single feature in isolation.
- Mention the evaluation numbers once here (38/38, 14/14) so the panel has them early.

Likely questions:

Q: Is this a real, deployed system or a prototype? A: It's a fully implemented, functionally complete MERN application, evaluated against a populated dataset (45 properties, 120 users, etc.) with real Razorpay and Gemini API integrations — not a mockup.

Slide 4 — Introduction *(0:45)*

Say: Property management has lagged behind banking and e-commerce in digital adoption. Operators still coordinate manually, causing double-bookings, unverifiable payments, and missed renewals — problems a unified, AI-assisted platform can remove.

Key points to emphasize:

- Connect "importance" directly to accessibility — the voice assistant point matters here, plant the seed early.
- Keep this slide brief; it's context-setting, not a result.

Slide 5 — Problem Statement *(0:40)*

Say: The core problem: no single system today covers listing through maintenance while also giving staff real-time awareness and giving buyers a cryptographic guarantee that their payment was actually recorded correctly.

Key points to emphasize:

- Read the 5 sub-bullets as one flowing problem, not a checklist.
- The "payment cannot be independently verified" point is your bridge to the security work later — flag it mentally.

Likely questions:

Q: Why is payment verification such a big deal for a student project? A: Because payment gateways return client-relayed data (order ID, payment ID, signature); if the server trusts that data blindly, an attacker can fake a successful payment. We close that by recomputing the HMAC-SHA256 signature server-side before touching any financial record.

Slide 6 — Objectives *(0:45)*

Say: Seven concrete objectives guided the design — from unifying the lifecycle to cryptographic payment verification to the multilingual voice assistant to Power BI analytics.

Key points to emphasize:

- Don't re-read all seven verbatim — group them out loud as "three themes: lifecycle unification, security, and accessibility/analytics."
- This slide is a good place to slow down slightly, since it maps 1:1 onto your later results slides.

Slide 7 — Scope of Work *(0:30)*

Say: To keep the system deployable and testable within this project's timeframe, native mobile apps, full UI localization, and IoT integration were placed in future scope rather than this phase.

Key points to emphasize:

- Frame exclusions positively — "deliberately scoped", not "didn't have time for".

Likely questions:

Q: Why no mobile app? A: The REST API is mobile-ready — a React Native client is explicitly listed as future work and would reuse the same backend without changes.

Slide 8 — Literature Review *(1:00)*

Say: I reviewed six representative papers across four themes: MERN architecture, LLM chatbots, multilingual voice assistants, and JWT security auditing, plus a systematic review of AI adoption in property management.

Key points to emphasize:

- Pick ONE row to speak about in a full sentence (recommend row 4, Sabharwal & Sahni on multilingual voice barriers — it's your strongest tie to the novel contribution) and just gesture at the rest.
- Do not read the whole table row by row — the panel can read it.

Likely questions:

Q: Why these six papers specifically? A: Each maps directly onto one design decision in the system: MERN architecture choice, chatbot/voice-assistant grounding, and JWT/payment security — they're not generic background, they're the specific gaps this project closes.

Slide 9 — Research Gaps *(0:45)*

Say: Three gaps emerge: architecture studies are evaluated in isolation, security mechanisms are audited as standalone tools, and no reviewed system combines domain-grounded, multilingual, voice-enabled assistance with verified payments in one tested platform.

Key points to emphasize:

- This slide is your thesis statement — say it slowly, it's the single most important sentence in the literature section.

Likely questions:

Q: What is genuinely novel here versus just "using existing APIs"? A: The novelty is the combination and the specific engineering choice: a zero-marginal-cost, browser-native voice interface layered onto a live-data-grounded chatbot — not any one library, but the integration and the accessibility outcome for Hindi/Telugu-speaking users, which the reviewed literature doesn't evaluate together.

Slide 10 — Existing System *(0:40)*

Say: Today's options are three kinds: pure manual coordination, browsing-only listing portals like Bayut or Dubizzle, and enterprise suites like Buildium or AppFolio that cost more and still lack verified real-time payments or AI/voice assistance.

Key points to emphasize:

- Name the disadvantages crisply — "manual, unverifiable, delayed, inaccessible, expensive" is a good five-word summary to fall back on.

Likely questions:

Q: Have you actually used Buildium/AppFolio to confirm they lack these features? A: This comparison is based on published feature documentation for these platforms, not hands-on enterprise trials — I'd frame it as a feature-level literature comparison rather than a hands-on benchmark.

Slide 11 — Proposed System *(0:50)*

Say: PropManage replaces that fragmented picture with one role-segmented, dual-portal platform: faster through real-time notification, more secure through cryptographic verification, and more accessible through a multilingual voice assistant.

Key points to emphasize:

- This is a good slide to make eye contact and slow down — it's your value-proposition summary before the technical deep-dive begins.

Slide 12 — System Architecture *(1:10)*

Say: The system follows this flow: a user interacts through the frontend, authenticates via JWT, can talk to the voice assistant, which — like every other action — goes through the backend APIs to MongoDB; the admin panel and Power BI reports draw from the same backend and database.

Key points to emphasize:

- Trace the diagram left-to-right, top row then bottom row, with your finger/pointer — don't just describe it from memory.
- Be ready to explain WHY authentication sits before the voice assistant in the flow (every request, including chat, is authorized).
- This is the single most-referenced slide if you get stuck later — you can always say "as shown in the architecture diagram" to re-anchor.

Likely questions:

Q: Why is voice assistant its own box instead of inside 'Backend APIs'? A: It's drawn separately because it's a distinct cross-cutting concern — a client-side Web Speech API layer plus a server-mediated Gemini call — not because it's a separate physical server; architecturally it's still part of the Node.js/Express application tier.

Q: Why MongoDB instead of a relational database? A: The domain has many interlinked but loosely structured entities (properties, bookings, contracts) that suit a document model, and MERN's single-JSON-representation across tiers reduces cross-layer transformation code — this is directly supported by the MERN architecture paper in the literature review.

Q: What happens if MongoDB Atlas goes down? A: That's a real single point of failure in the current design — noted honestly, not hidden. Mongoose reconnection with backoff is implemented, but full high-availability failover is out of scope for this phase.

Slide 13 — Methodology / Algorithms *(1:30)*

Say: Five mechanisms do the real work: JWT authentication with per-request role re-checking, HMAC-SHA256 payment verification, WebSocket real-time notification, the multilingual voice assistant pipeline, and the Power BI aggregation pipeline.

Key points to emphasize:

- This is your longest, densest slide — rehearse it separately until you can say each "Purpose / Working / Advantage" triple in under 15 seconds.

- If time is short, you may compress algorithms 3 and 5 to one sentence each and spend the saved time on 2 and 4 (payments and voice), which draw the most questions.

Likely questions:

Q: Why HMAC-SHA256 specifically, and not just checking the payment status field? A: Because a status field can be reported by the client and isn't cryptographically bound to anything — HMAC-SHA256 recomputes a signature server-side over the order and payment identifiers using a secret only the server holds, so a mismatch proves tampering rather than merely disagreeing with a client claim.

Q: Why Gemini instead of GPT or Claude? A: Gemini was the provider actually integrated and load-tested in this build; the architecture is provider-agnostic at the prompt-assembly layer, so swapping providers would not require redesigning the pipeline.

Q: Does the voice assistant work fully offline? A: No — speech recognition and synthesis are browser-native (Web Speech API, no server round-trip for the audio itself), but the conversational reply still requires an internet connection to reach the Gemini API.

Q: Is this true Retrieval-Augmented Generation (RAG)? A: Not yet — today it's a lighter-weight context-assembly approach: a live database snapshot is injected into the prompt. Migrating to embedding-based retrieval over a vector index is explicitly named as future work.

Slide 14 — Working Modules *(0:35)*

Say: Seven modules divide responsibility cleanly: user, authentication, voice assistant, booking & payment, admin, database, and reporting.

Key points to emphasize:

- Just name the seven modules in one breath — this slide is a summary, not a place to re-explain each one (you already did that in Algorithms).

Slide 15 — Requirements *(0:25)*

Say: Hardware requirements are modest — a dual-core machine with 4GB RAM — and the software stack is entirely free-tier: Node.js, React, MongoDB Atlas, and public APIs.

Key points to emphasize:

- Emphasize "free-tier" — it directly supports the cost-accessibility claim from the Abstract and Conclusion.

Slide 16 — Technology Stack *(0:40)*

Say: Frontend and backend use React and Node/Express with MongoDB and Socket.IO for real-time; third-party services handle payments, AI/voice, storage, and analytics.

Key points to emphasize:

- If asked to justify any single technology choice, tie it back to a requirement (Socket.IO -> real-time objective; Razorpay -> verified-payment objective).

Likely questions:

Q: Why Razorpay specifically? A: It's a widely used gateway with strong signature-verification documentation and INR support relevant to the target market, and it was the gateway actually integrated and tested in this build.

Slide 17 — Experimental Results *(1:00)*

Say: Testing used a populated dataset of 45 properties and 120 users. All 38 functional test scenarios passed, all 14 adversarial security checks passed, and the chart shows endpoint latency across the system's core operations.

Key points to emphasize:

- Point at the AI Chat bar specifically and explain why it's the slowest (external LLM round-trip) before anyone asks.
- State the 100-concurrent-user / sub-500ms result even though it's on the next slide's table — panels often ask about load here.

Likely questions:

Q: Was this tested on real users or synthetic data? A: Synthetic but realistically scaled data (45 properties, 120 users, 280 bookings, etc.) seeded specifically to reflect production query complexity, plus scripted adversarial test cases — not live production traffic.

Q: 2080ms for the AI endpoint seems slow — is that a problem? A: It reflects the external Gemini API round-trip for a streamed conversational response, which is normal and acceptable for chat-style interaction, not a page-load-critical path.

Slide 18 — Execution *(0:30)*

Say: Here's a walkthrough of the live system: login, dashboard, the voice assistant in action, and the admin panel.

Key points to emphasize:

- Narrate the USER JOURNEY across the four screenshots as one story, not four disconnected captions.
- If a screenshot is still a placeholder on viva day, say so plainly and move on — don't apologize excessively.

Slide 19 — Result Analysis *(0:45)*

Say: Summarizing: 100% functional and security pass rates, sub-500ms mean latency at 100 concurrent users, and real-time notification delivery under two seconds in field observation.

Key points to emphasize:

- "100%" is a strong word — be ready to immediately qualify it with the exact denominators (38/38, 14/14) so it doesn't sound inflated.

Likely questions:

Q: 100% security pass rate — does that mean the system is fully secure? A: It means all fourteen specific adversarial checks that were designed and tested passed — it's evidence against those specific threat categories (JWT tampering, injection, signature tampering, etc.), not a claim of exhaustive security coverage against every possible attack.

Slide 20 — Conclusion *(0:40)*

Say: PropManage delivers a dual-portal MERN platform meeting all five objectives, empirically validated, and built entirely on free-tier tooling — proof that a small operator doesn't need an enterprise budget for this capability.

Key points to emphasize:

- This is your "landing" slide — deliver it with confidence and slightly slower pace than the rest of the talk.

Slide 21 — Future Scope *(0:35)*

Say: Six directions extend this work: full RAG-based AI grounding, a native mobile app, complete UI localization, predictive analytics, fault-tolerant analytics aggregation, and IoT integration.

Key points to emphasize:

- If asked "what would you do with one more month," answer from this slide — RAG migration or predictive analytics are the strongest, most defensible picks.

Likely questions:

Q: Which of these would you personally prioritize? A: Predictive analytics and RAG-based retrieval — both build directly on data and infrastructure the system already has (booking/payment history, MongoDB), so they extend rather than replace the current architecture.

Slide 22 — References *(0:15)*

Say: These are the fifteen sources underpinning the literature review and design decisions, spanning IEEE, Springer, Elsevier, and MDPI publications from 2023-2026.

Key points to emphasize:

- Do not read citations aloud — just state the range/venues in one sentence, as shown.

Slide 23 — References (contd.) *(0:15)*

Say: Continued reference list.

Key points to emphasize:

- Skip past quickly — say "and the remainder of the reference list" while advancing.

Cross-Cutting Panel Questions (Not Tied to One Slide)

Q: What is the single biggest technical challenge you faced?

A: Getting payment verification right — the first instinct is to trust the client's reported payment status, and it took real security analysis to recognize that only a server-side HMAC recomputation against the gateway's own signature actually closes that gap.

Q: If you had to cut one feature to save time, which would it be and why?

A: Power BI analytics — it's valuable but additive; the core lifecycle, security, and voice-assistant work would still stand as a complete system without it.

Q: How is this different from just calling three APIs (Razorpay, Gemini, Power BI) and gluing them together?

A: The engineering value is in the verification logic around each integration — recomputing signatures rather than trusting responses, scoping real-time events to authenticated rooms, and isolating the BI API behind its own non-expiring credential — not the API calls themselves.

Q: What would break first if you had 10,000 users instead of 120?

A: The single parallelized query batch behind the admin dashboard — it currently fails the whole dashboard request if any one of its eight queries fails, and would need to be made independently fault-tolerant, which is explicitly named as future work.

Q: Did you write all of this code yourself?

A: Answer honestly and specifically about your own role and any tools or assistance used, consistent with your institution's academic integrity policy — this guide cannot answer that for you.